

AVALANCHE FUJI · CHAIN 43113 · LIVE

Ward

On-chain action control for autonomous actors.

One markdown policy. One Solidity modifier. Your agent, keeper, or treasury bot can only call the contracts and functions you allow — **enforced in the same transaction**. It complements Kite's control of *how much* an agent spends with control of *what* it calls.

• Live & verified on Fuji

• 149 Foundry tests, 0 failing

• Policy semantics modeled in Lean 4

Keepers, DAO treasury bots, delegated vault managers — and the arriving wave of AI agents — already hold signing authority over real funds on Avalanche. That is authority handed to software that can be **tricked, compromised by a prompt injection, or simply wrong**.

TODAY

Keepers & automation

Liquidation bots, upkeeps, and Safe-based treasury ops run unattended.

TODAY

Delegated DeFi

Vault managers move capital across ~\$2B in Avalanche TVL.

EMERGING

AI agents

LLM-driven actors that emit executable calldata — the loudest bet on the chain.

Today's controls bound how much leaves — not what gets called.

Kite's spend caps and Safe's spending modules cap value: "under \$100/day."

Necessary — but they govern the wrong axis. A spend cap says nothing about *which contracts and functions* an actor may touch.

A \$100 CAP DOESN'T STOP...

\$50 → a phishing contract

Well under the cap. Straight out the door.

A \$100 CAP DOESN'T STOP...

wrongFn() on a real contract

Right destination, wrong call.

A \$100 CAP DOESN'T STOP...

approve(drain, MAX)

Zero value moves in the call. The theft happens later.

Declare what your actor may do. Enforce it on-chain.

1

Write **POLICY.md**

Plain markdown — allow-listed contracts, function selectors, value caps, and delay/veto tiers.

2

Compile → **policyId**

A deterministic hash. Auditable, diffable, versionable. Publish it to `WardOracle`.

3

Add one modifier

The agent's own call is checked against the policy — in the same transaction, before it executes.

```
// one line guards the actor's own outbound call
contract MyAgent is WardAgentBase {
    function swap(address pool, uint256 amt)
        external payable
        wardGuarded(this.swap.selector, msg.value)
    {
        // runs only if the call passes policy
    }
}
```

No custody. No execution. No platform to adopt. `WardOracle` is a view — it never holds funds, owns an agent, or makes a call.

Honest about what's a hard block, what's contained, and what's next.

BLOCKED SYNCHRONOUSLY

- ◆ Call to a non-allow-listed contract
- ◆ Disallowed function selector
- ◆ Native value over per-call / daily cap
- ◆ Paused or expired policy
- ◆ `setPolicyId(0)` — instant kill switch

CONTAINED · TIERS + VETO

- ◆ A sensitive selector (e.g. `approve`) set to `DELAYED` / `VETO_REQUIRED`
- ◆ Routes through `WardQueue`
- ◆ Operator is alerted and vetoes within the delay window

ON THE ROADMAP

◇ Argument-level constraints

◇ Pin an approve spender

COMPLEMENTARY, NOT COMPETITIVE

◇ Cap ERC-20 amounts — upgrades the approve case from vetoed to auto-blocked

Kite controls how much. Ward controls what.

KITE · PAYMENTS

How much an agent spends

Spend caps, per-tx approval thresholds — a platform you adopt for agent payments. Live on mainnet, \$35M raised.

+

WARD · ACTIONS

What an actor may call

Target + selector policy, in-transaction, no custody. Drops into any C-Chain actor — including agents already using Kite for payments.

A Kite agent with a \$100/day cap can still be tricked into an unlimited approval. Ward closes that hole. Together they bound an actor on both axes — the layer the agent stack still lacks.

MARKET

Real users today. The agent wave next.

LIVE MARKET • DAY ONE

Keepers, treasury bots, DeFi automation

Autonomous actors already run across Avalanche's ~\$2B DeFi TVL — liquidation keepers, vault managers, and Safe-based treasury ops that need action bounds now.

GROWTH CURVE

The autonomous agent economy

Avalanche's loudest forward narrative — **Kite** live on mainnet, **ERC-8004** identity, **x402** settlement. Ward rides in as it scales.

Same primitive, same modifier, both audiences. Agents are the story; automated actors are the day-one users.

On-chain action scoping has incumbents. Ward owns the gap they can't.

THE INCUMBENTS

- ◆ **Zodiac Roles Modifier** — on-chain, argument-level scoping today, on Avalanche. But it guards transactions routed *through a Safe*.
- ◆ **ERC-7579 smart sessions** — real on-chain policy, for smart-account wallets.

WARD'S NICHE — ACTOR-AGNOSTIC

- ◆ Guards a **plain agent / keeper contract's own outbound calls** — no Safe, no smart-account wrapper required
- ◆ Declarative `POLICY.md`, one modifier
- ◆ Tiers + delay + veto integrated with the policy
- ◆ A machine-checked semantic model

We don't out-feature Zodiac on argument scoping today. We occupy the actor-contract niche it doesn't — with a lower integration surface.

NOT A PLAN – SHIPPED

Live, verified, and tested on Avalanche today.

3

CONTRACTS LIVE & SOURCE-VERIFIED ON FUJI

10

LEAN 4 THEOREMS, ZERO SORRY

WardOracle	0x111C0Eb8e964f8fE2725d5Bbeb9E1c41CAE2093E
WardQueue	0x9d3352f46B9dd2b3705449F69BA994f013cf5C26
WardAgentRegistry	0x821427e8fc8d55737d210E868C485409256A9815

149

FOUNDRY TESTS, 0 FAILING (INCL. INVARIANTS)

Full

STACK: CONTRACTS · SDK · CLI · DASHBOARD

The engineering risk is retired. Lean models the policy semantics; property tests bind the Solidity to that model – stronger than tests alone, and rare at this stage.

WHAT'S NEXT

The grant funds adoption — not construction.

M1 · MAINNET + BENCHMARKS

Promote the verified contracts to C-Chain; publish measured per-guarded-call gas at real fees.

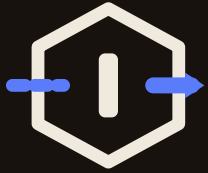
M2 · REFERENCE ACTORS

A selector-restricted DEX swapper and a treasury/keeper bot, live — one demoed alongside a Kite payment flow.

M3 · REVIEW + FIRST INTEGRATOR

A scoped independent security review, and the first external design partner running Ward in production.

The code is written, tested, and live. What the grant buys is the part code can't: an audited, benchmarked mainnet deployment and the first builders using it.



Bind what your agent can do.

One markdown policy. One modifier. In the same transaction.

github.com/oderahub/WARD

• Live on Avalanche Fuji • chain 43113